# Operational semantics for Prolog with Cut in Rocq and its application to determinacy analysis

**Davide Fissore**
Université Côte d'Azur, Inria, France

**Enrico Tassi**
Université Côte d'Azur, Inria, France

**Abstract**

We mechanize two operational semantics for Prolog with cut and prove them equivalent, in the Rocq prover. The first semantics closely models the Elpi's implementation, it is based on a stack of choice points and is well suited for studying optimizations employed by Prolog abstract machines. The second semantics is instead based on and-or trees, in which the branches pruned by the cut operator are made explicit.

We use the tree-based semantics (1) to prove properties about the effect of the cut operator in the evaluation of choice points, since the rules from classical logic do not apply, and (2) to reason about the determinacy analysis introduced in Elpi version 3.0, which statically identifies computations that produce at most one result.

**2012 ACM Subject Classification** Theory of computation → Operational semantics

**Keywords and phrases** Semantics, Prolog, Elpi, Determinacy Analysis, Rocq, Coq

**Digital Object Identifier** 10.4230/LIPIcs.CVIT.2016.23

**Funding** *Davide Fissore*: This work has been supported by the French government, through the France 2030 investment plan managed by the Agence Nationale de la Recherche, as part of the "UCA DS4H" project, reference ANR-17-EURE-0004.

## 1 Introduction

Elpi is a dialect of $\lambda$Prolog (see [14, 15, 7, 12]) used as an extension language for the Rocq prover (formerly the Coq proof assistant). Elpi has become an important infrastructure component: several projects and libraries depend on it [13, 3, 4, 19, 8, 9]. Examples include the Hierarchy-Builder library-structuring tool [5] and Derive [17, 18, 11], a program-and-proof synthesis framework with industrial applications at SkyLabs AI.

Starting with version 3, Elpi is equipped with a static analysis for determinacy [10] to help users tame backtracking. Rocq users are familiar with functional programming but not necessarily with logic programming, in particular uncontrolled backtracking is a common source of inefficiency and makes debugging harder. The determinacy checker identifies predicates that behave like functions, i.e., predicates that commit to their first solution and leave no *choice points* (places where backtracking could resume).

This paper reports our first steps towards a mechanization, in the Rocq prover, of the determinacy analysis from [10]. We focus on the control operator *cut*, which is useful to restrict backtracking but makes the semantics depart from a pure logical reading.

We formalize two operational semantics for Prolog with cut. The first is a stack-based semantics that closely models Elpi's implementation and is similar to the semantics mechanized by Pusch in Isabelle/HOL [16] and to the model of Debray and Mishra [6, Sec. 4.3]. This stack-based semantics is a good starting point to study further optimizations used by standard Prolog abstract machines [20, 1], but it makes reasoning about the scope of cut difficult. To address this limitation we introduce a tree-based semantics in which the branches pruned by cut are explicit and we prove the two semantics equivalent. Using the

```
Inductive P := IP of nat. Inductive D := ID of nat. Inductive V := IV of nat.

Inductive Tm :=
  | Tm_P of P       | Tm_D    of D
  | Tm_V of V       | Tm_App  of Tm & Tm.
```

**Figure 1** Tm types

```
Record Unif := {
  unify : Tm → Tm → Σ → option Σ;
  matching : Tm → Tm → Σ → option Σ;
}.
```

**Figure 2** Unif type

tree-based semantics, we then prove some properties about the impact of evaluating the cut in disjunctive querys. For example, unlike in classical logic, $q_1 \vee q_2$ is not equivalent to $q_2 \vee q_1$, since $q_2$ may be cut-away by $q_1$. Moreover, we show that if every rule of a predicate passes the determinacy analysis, then a call to that predicate does not leave any choice points.

## 2 Preliminaries: syntax and informal semantics

put unif and program in variables hides from types

Before going to the two semantcis, we show the piece of data structure that are shared by the them. The smallest unit of code that we can use in the langauge is an atom. The atom inductive (see Definition 1) is either a cut or a call. A call carries a term (see Figure 1). A term (Tm) is either a predicate, a datum, a variable or the binary application of two terms.

```
Inductive A := cut | call : Tm → A.                                (1)
Record R := mkR { head : Tm; premises : list A }.                  (2)
Record ℙ := { rules : seq R; sig : sigT }.                         (3)
Definition Σ := {fmap V → Tm}.                                     (4)
Definition bc : ℙ → 𝓕𝓿 → Tm → Σ → 𝓕𝓿 * seq (Σ * seq A) :=          (5)
```

se riesco extract lemma immediately with lemma environment

A rule (see Definition 2) has a head of type term and a list of premises, that are a list of atoms. A program (see Definition 3) is made by its rules and the signatures of predicates, `sigT` is a mapping from predicates to their signatures, signatures represent the defintion of type from the simply typed lambda calculus, i.e. it is either a base type or an arrow for term application. We decorate arrows to know the mode (input or output) of the application argument.

A substitution (see Definition 4) is a mapping from variables to terms. It is the output of a successful query. The empty substitution is noted $\varepsilon$.

The backchain function (bc, see Definition 5) takes: a program $P$; a set $fv$ of variables ($\mathcal{F}_v$); a query $q$; and the substitution $\sigma$. A backchain is performed when the current atom is a `call`. In this case, it start by recursively *dereferencing* the variables in $\sigma$, i.e. replacing all the variables in the query by their term mapping in $\sigma$.

TODO: free variables and program freshness

Then it takes the head of the term, which should be rigid, i.e. a predicate $p$, and look for the signature $s$ of $p$ in the program. Then, for each rule $r$ in $P$, it unifies the head of $r$ with $q$. Unification is possible thanks to the unifcator $U$, whose type is shown in Figure 2. $U$ explains how to unify terms up to standard unifcation (for output terms) or matching (for input terms). The function *unif* (resp. *matching*) unifies (resp. matches) the two terms under $\sigma$, it return an extension of $\sigma$ if the two terms can be unified (resp. matched), and None otherwise. The result of a backchain is made by filtering the rules in $P$ that can be used on the given query. In particular, for each rule filtered rules $r$, it returnes a pair containing the extension of $\sigma$ making the head of $r$ and $q$ unify and the list of premises of $r$.

```
f 1 2.   f 2 3.
r 0 0.   r 0 1.   r 0 2 :- !.
g A C :- r A B, f B C, !.  % r1
g D F :- f D E, f E F.     % r2
```

**Figure 3** Small ELPI program example

```
Inductive tree :=
  | KO | OK | TA of A
  | Or  of option tree & Σ & tree
  | And of tree & seq A & tree.
```

**Figure 4** Tree inductive

## 2.1 The cut operator

ELPI is a dialect of λPROLOG with the hard cut operator and, in the language we have shown before, *cut* is an atom. The semantics of the cut operator adopted in the ELPI language corresponds to the *hard cut* operator of standard SWI-PROLOG. This operator has two primary purposes. First, it eliminates all alternatives that are created either simultaneously with, or after, the introduction of the cut into the execution state.

The ELPI program we will use as an example in the following sections is shown in Figure 3.

To illustrate the high-level description of *cut*, consider the program $P$ shown in Figure 3 and the query $q = \texttt{g 0 R}$. Both rules for `g` can be used on the query $q$. Backchaining $q$ in $P$ from the empty ($\varepsilon$) subsitution returns the list $[(\sigma_1, [\texttt{r A B, f B C, !}]), (\sigma_2, [\texttt{f D E, f E F}])]$ with $\sigma_1 := \{\texttt{A} \mapsto \texttt{0}, \texttt{B} \mapsto \texttt{C}\}$ and $\sigma_2 := \{\texttt{D} \mapsto \texttt{0}, \texttt{F} \mapsto \texttt{R}\}$. The two elements of the list are generated respectively from the rules $r_1$ and $r_2$. For simplicity, we have taken a program where the variable of each pair rule are disjoint and a query with variables disjoint from the program, this avoid the *bc* function to fresh the program variable.

The next step of the interpretation corresponds to the backchaining of `r X Y` from $\sigma_1$. This returns $[(\sigma_3, []), (\sigma_4, []), (\sigma_5, [\texttt{!}])]$ with $\sigma_3 := \sigma_1 + \{\texttt{B} \mapsto \texttt{0}\}$, $\sigma_4 := \sigma_1 + \{\texttt{B} \mapsto \texttt{1}\}$ and $\sigma_5 := \sigma_1 + \{\texttt{B} \mapsto \texttt{2}\}$. The three lists of premises are empty since all rules for `r` have no premises.

Backchaining `f B C` in $\sigma_3$ gives no solution, therefore, by backtracking, we backchain `f B C` in $\sigma_4$. This gives a solution, namely $\sigma_6 := \sigma_4 + \{\texttt{R} \mapsto \texttt{2}\}$.

We now reach the cut operator and can see its double side behavior. In the current execution state, we have still two unexplored alternatives: we can backtrack on the third solution for `r` that assign `B` to `2` and on the rules $r_2$. We can note, however, the first alternative have been generated after the cut introduction, and the second alternative have been generated at the same moment of the cut. Therefore both are cut away leaving no choice point. The final complete solution is: $\{\texttt{A} \mapsto \texttt{0}, \texttt{B} \mapsto \texttt{1}, \texttt{C} \mapsto \texttt{R}, \texttt{R} \mapsto \texttt{2}\}$, where `A`, `B` and `C` can be ignored since they are the result internal unification.

We propose two formal, operational semantics for a logic program with cut. The two semantics are based on different syntaxes, the first syntax (called tree) exploits a tree-like structure and is ideal both to have a graphical view of its evolution while the tree is being intepreted and to prove lemmas over it. The second syntax, called elpi, is the ELPI's syntax and has the advantage of reducing the computational cost of cutting and backtracking alternatives by using shared pointers. We aim to prove the equivalence of the two semantics together with some interesting lemmas of the cut behavior.

*Notations used in the paper* In the following section we note $\mathbb{B}$ for the bolean type, and $\top$ and $\bot$ for `true` and `false`. The option instances are noted $\bullet t$ for *Some* $t$ and $\square$ for *None*.

spiegare notation for list

```
Fixpoint get_end σ A : Σ * tree:=
  match A with
  | TA _ | KO | OK ⇒ (σ, A)
  | Or □ s1 B ⇒ get_end s1 B
  | Or •A _ _ ⇒ get_end σ A
  | And A _ B ⇒
    let (σ', pA) := get_end σ A in
    if pA == OK then get_end σ' B
    else (σ', pA)
  end.
```

**(a)** Defintion of *get_end*

```
Definition get_subst σ A := (get_end σ A).1.

Definition path_end A := (get_end ε A).2.

Definition success A := path_end A == OK.

Definition failed A := path_end A == KO.

Definition path_atom A :=
    if path_end A is TA _ then ⊤
    else ⊥.
```

**(b)** Functions from *get_end*

**Figure 5** *get_end* and derives functions

## 3 And-Or Tree semantics

*The tree state* The tree inductive, shown in Figure 4. We distinguish 5 main cases: *KO*, *OK*, and are special meta-symbols representing, respectively, the basic failed and a successful trees. These symbols are considered meta because they are internal intermediate symbols used to give structure to the tree.

The *TA* constructor (acronym for tree-atom) is the constructor of atoms in the tree. They are either the cut operator or a call, in these cases, the interpreter should evaluate the atom by either cutting the subtrees in the scope of the cut or backchaining the call in the program.

The two recursive cases of a tree are the *Or* and *And* nodes. The *Or* node $A \vee B_\sigma$ denotes a disjunction between two trees $A$ and $B$. The first branch is optional, if absent it represents a dead tree, i.e. a tree that has been entirely explored. The second branch is annotated with a suspended substitution $\sigma$ so that, upon backtracking to $B$, $\sigma$ is used as the initial substitution for the execution of $B$.

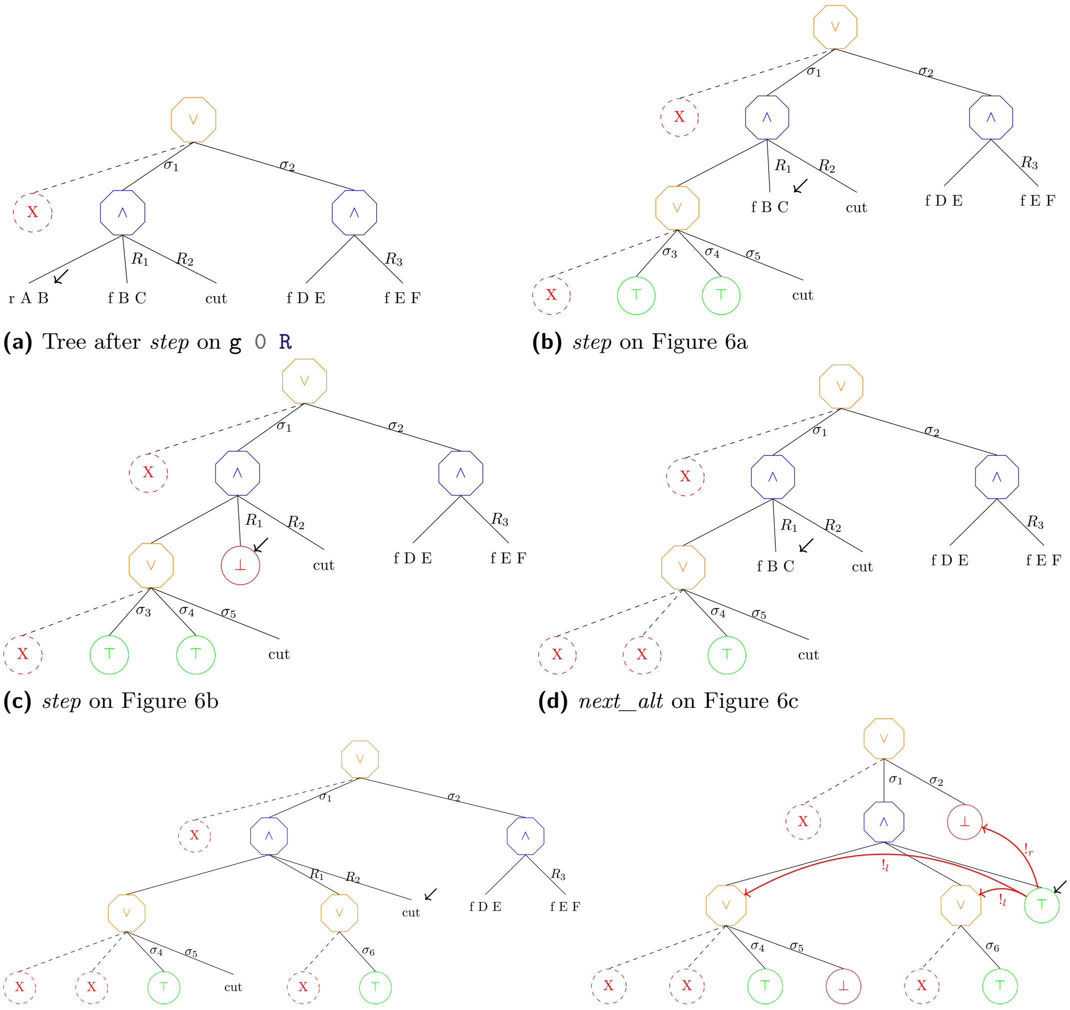
The *And* node $A \wedge_{B_0} B$ represents a conjunction of two trees $A$ and $B$. We call $B_0$ the reset point for $B$; it is used to restore the state of $B$ to its initial form if a backtracking operation occurs on $A$. An example of this behavior is shown in Section 3.2

A graphical representation of a tree is shown in Figure 6a. To make the graph more compact, the *And* and *Or* nodes are n-ary rather than binary, and the children of these nodes associate to the right. The *KO* node acts as the neutral elements in the *Or* list, while *OK* is the neutral element of the *And* list.

*The tree interpreter* The interpretation of a tree is performed by two main routines: *step* and *next_alt* that traverse the tree depth-first, left-to-right. Then, then *runT* inductive makes the transitive closure of step *step* and *next_alt*: it iterates the calls to its these auxiliary functions as much as needed. In Definitions 7–9 we give the types contrats of these symbols where $\mathcal{F}_v$ is a set of variable names.

```
Inductive tag := Expanded | CutBrothers | Failed | Success.                  (6)
Definition step : ℙ → 𝓕v → Σ → tree → (𝓕v * tag * tree) :=                    (7)
Definition next_alt : 𝔹 → tree → option tree :=                               (8)
Inductive runT (p : ℙ): 𝓕v → Σ → tree
          → option (Σ * option tree) → 𝔹 → 𝓕v → Prop :=                        (9)
```

The tree interpreter, as in prolog, explores the tree in DFS strategy, to discover the substitution and the leaf of the tree that should be interpreted. These two pieces of information can be retrieved for a substitution $s$ and a tree $A$ thanks to the *get_end* routine,




**(a)** Tree after *step* on `g 0 R`

**(b)** *step* on Figure 6a

**(c)** *step* on Figure 6b

**(d)** *next_alt* on Figure 6c

**(e)** *step* on Figure 6d

**(f)** *step* on Figure 6e

**Figure 6** Some tree representations[1]

shown in Figure 5a If the tree is already a leaf, *get_end* returns its inputs. Otherwise, if the tree is a disjunction, the path continues on the left subtree, if it exists, otherwise *get_end* contues on the ths using the substitution stored in the *Or* node. In the case of a conjunction, if *get_end* of the lhs returns the tree *OK*, then the *get_end* continues in the rhs, otherwise we return the result of the lhs. We derive some useful definition from *get_end*. They are listed in Figure 5b.

We have pointed by a black arraw the *path_end* of all the trees in Figure 6.

fare fresh rules in Figure 6

## 3.1 The *step* procedure

The *step* procedure takes as input a program, a set of variables, a substitution, and a tree, and returns an updated set of variables, a *tag*, and an updated tree.

[1] The substitution $\sigma_1 \ldots \sigma_6$ are from Section 2.1, $R_1, R_2$ and $R_3$ are the reset points and correspond respectively to the list of atoms [`f Y Z, !`], [`!`] and [`f Y Z`]

The set of variables $v$ given in input to step are used and updated when a backchaining operation takes place. A call to *step* produces a good result if $v$ contains all the variables in the tree. This guarantees that any call to *bc* returns a list of atoms whose variables are disjoint from the variables of the current working tree.

not true in figure:fix + also fix in Section 2.1

The *tag* (see Definition 6) indicates the kind of an internal tree step: `CutBrothers` denotes the interpretation of a superficial cut, i.e., a cut whose parent nodes are all *And*-nodes. `Expanded` denotes the interpretation of non-superficial cuts or predicate calls. `Failure` and `Success` are returned for, respectively, *failed* and *success* trees.

The step procedure is intended to evaluate atoms, that is, it leaves unchanged the tree iff its *path_end* is not an atom, i.e. if it is a success- or a failed-tree.

`Lemma succ_step_iff: ∀ p v σ A, success A ↔ step p v σ A = (v, Success, A).` (10)

`Lemma fail_step_iff: ∀ p v σ A, failed A ↔ step p v σ A = (v, Failed, A).` (11)

*The interpretation of a call* to a term $c$ stars by calling the *bc* function on $c$. The result of the backchain is a list $l$. If $l$ is empty then the current call is replaced by the *KO* node, otherwise it is replaced by a right-skewed tree made by *Or* inner-nodes and each leave correspod to a list of atom $e$ in $l$. If $e$ is empty then it is mapped to the *OK* tree, otherwise it is mapped to a right-skewed tree made by *And* inner-nodes.

sottolineare le sostituzioni

In Figure 6a, we provide a clear example of a call to *step* on the query `q 0 R` on the program $P$ in Figure 3. Let $c_0$, $c_1$ and $c_2$ be the childrens of the root. $c_0$, by construction is the *None* tree, while $c_1$ and $c_2$ correspod respectively to the premises of rules $r_1$ and $r_2$ in $P$. We need the first *None* subtree so that we can attach a substituion to $c_1$. In particular, we note that the $c_1$ (resp. $c_2$) are related to the substitution $\sigma_1$ (resp. $\sigma_2$). The value of $\sigma_1$ and $\sigma_2$ are the same as in Section 2.1. The reset points in the tree are marked with the $R$ symbol. $R_1 :=$ [`f Y Z, !`], $R_2 :=$ [`!`] and $R_3 :=$ `f Y Z`, they are essentially the list of atoms that allowed to generate the current subtree. Other example of call to the *step* procedure triggering a backchain are Figures 6b, 6c, and 6e.

*The interpretation of a cut* has three main impacts: at first the cut is replaced by the *OK* node, then some special subtrees, in the scope of the *Cut*, are cut away: in particular we need to soft-kill the left-siblings of the *Cut* and hard-kill the right-uncles of the the *Cut*.

▶ **Definition 1** (Left-siblings (resp. right-sibling) and Right-uncles)**.** *Given a node $A$, the left-siblings (resp. right-sibling) of $A$ are the list of subtrees sharing the same parent of $A$ and that appear on its left (resp. right). The right-uncles of $A$ are the list of right-sibling of the father of $A$.*

▶ **Definition 2** (Hard-kill, $!_r$ and Soft-kill, $!_l$ )**.** *Given a tree $t$, hard-kill replaces the given subtree with the* KO *node. If $t$ is a success-tree, then soft-kill replaces with* KO *all subtrees that are not part of the path in $t$ leading to the* OK *node.*

An example of the impact of the cut is show in Figure 6f. In Figure 6e, we see that the *path_end* is a cut. The cut is replaced by the *OK* node and it need to cut-away some alternatives in the tree. The red arrows indicate the scope of the cut, they are labeled with $!_r$ and $!_l$ to explain which kind of cut has been performed on each pointed subtree. We note that, after the evaluation of a cut, the path leading to the cut node is not affected by $!_l$ , the success under the substitution $\sigma_4$ is kept, while the success under the substitution $\sigma_5$ is replaced by the *KO* node. If we refer to the paragraph in Section 2.1, we are discarding the third rule of the `r` predicate.

## 3.2 The *next_alt* procedure

The *step* procedure alone is not sufficient to reproduce entirely the behavior of the full expected prolog interpreter. As proved in Definition 10 and Definition 11, *step* does not make any progression on the tree being interpreted. To solve this problem, we need a routine capable of backtracking in case of failures. This routine return the backtracked tree if it exists, *None* otherwise.

In Figure 6c, we are facing a failure: the interpreter has evaluated the atom `f X Y` under the substitution $\sigma_3$ which assigns `Y` to `0`, but, in Figure 3, we have no rule satisfying this query. The role of *next_alt*, in this example, is to kill the path leading to this failure, while reseting the failure to its original shape. We see how the reset point plays a major role in this case. At first we are killing the *OK* node under the subsitution $\sigma_3$, since this tree have been entirely explored and led to no solution, then we replace the *KO* node with the information stored in $R_1$. That is, thanks to $R_1$, we are restoring the `f Y Z` call, so that we are able to evaluate this node under the subsitutution $\sigma_4$. More concretely, we have uncessefully tried rule `r 0 0`, therefore, we need to continue by trying rule `r 0 1`.

Furthermore, *next_alt* accomplishes to another important task. Its behavior changes depending to the value of the bolean it receives in entry: next_alt *false* $A$ aims to recursively cancel all the failures (if any) in $A$, whereas, next_alt *true* $A$ aims to cancel the first success in $A$. This second behavior is helpful since, we need to progress the tree in case of success. For example, consider the tree in Figure 6f, it contains a success, mapping namely `R` to `2`. The interpreter should return this sucess, but, due to the "non-deterministic" behavior of the prolog system, we also need to return all the other solution that could be available by evaluating the rest of the tree. After the replacement of the *path_end* in Figure 6f with *KO*, we remain with a tree having all failing path. The *next_alt* procedure keeps removing them until discovering that the tree have no worth-to explore paths, therefore it will return *None*.

Some interesting property of *next_alt* are shown below and allow to see how *next_alt* complements *step* wrt failed, success and *path_atom* trees.

```
Lemma path_atom_next_alt_id: ∀ b A, path_atom A → next_alt b A = •A.        (12)
Lemma next_alt_failedF: ∀ b A A', next_alt b A = •A' → failed A' = ⊥.       (13)
Lemma next_altFN_fail: ∀ A, next_alt ⊥ A = □ → failed A.                    (14)
```

## 3.3 The *runT* inductive

The inductive *runT* is modeled as a function taking 4 inputs and returning 3 outputs. The inputs are the program, a set of variables $v_0$, an initial substitution $\sigma_0$, and a tree $t_0$. The outputs are an optional object $r$ containing a substitution $\sigma_1$ and an optional tree $t_1$, and a boolean $b$ and a set of updated variables $v_1$. The very interesting piece of information is the result $r$, if *None*, it tells that the interpretation failed, otherwise, $\sigma_1$ represents the most-general unificator that makes the execution of the tree $t_0$ under the initial substitution $\sigma_0$ succeeds: $\sigma_1$ is an extension of $\sigma_0$. The tree $t_1$ is the updated tree containing the alternatives that have not yet been explored, if these alternatives do not exists, $t_1$ is *None*. The output $b$ and $v_1$ are there to tell respectively if during the execution a superficial cut has been evaluated, and what is the updated set of variables after the interpretation.

The procedure *runT* is based on four main derivation rules, shown in Figure 7. (StopT) If the *path_end* of the tree $t_0$ is a success, the substitution $\sigma_1$ is retrived in $t_0$ from $\sigma_0$ and the input tree is cleaned of its successful path. (StepT) If the *path_end* of $t_0$ is an atom, then *step* is invoked to evaluate $t_0$, and *runT* is recursively called on the new tree. (BackT)

If the *path_end* of $t_0$ is a failure, *next_alt* is called to clear the failed path; if the resulting cleaned tree $t_1$ exists, *runT* is recursively called on it. Finally, (FAILT) takes into account tree with no solutions.

$$\begin{array}{l}\textbf{Lemma}\ \texttt{runT\_det}:\ \forall\ \texttt{p}\ v_0\ \texttt{s0}\ t_0\ r_1\ r_2\ b_1\ b_2\ v_1\ v_2,\\ \quad \texttt{runT p}\ v_0\ \texttt{s0}\ t_0\ r_1\ b_1\ v_1 \to \texttt{runT p}\ v_0\ \texttt{s0}\ t_0\ r_2\ b_2\ v_2 \to r_2 = r_1 \wedge v_2 = v_1 \wedge b_2 = b_1.\end{array} \tag{15}$$

The lemma above tells that *runT* behaves deterministically. The proof is by induction on the first *runT* execution and the proof is rather immediate, since all rules are in exclusive due to the hypothis on the *path_end*: the hypotheses *success* $t_0$, path_atom $t_0$ and *failed* $t_0$ are exclusive. Moreover, by Definition 15, we have that the hypothesis of FAILT implies *failed* $t_0$, therefore it is exclusive wrt STOPT and STEPT, but also from BACKT since the have different expectations on the output of *next_alt*.

$$\frac{\text{success } t_0 \qquad \text{get\_subst s0 } t_0 = \text{s1} \qquad \text{next\_alt } \top\ t_0 = t_1}{\text{runT } v_0 \text{ s0 } t_0\ \bullet(\text{s1}, t_1)\ \bot\ v_0}\ \textsc{StopT}$$

$$\frac{\begin{array}{cc}\text{path\_atom } t_0 & b_2 = \text{is\_cb tg} \mid\mid b_1\\ \text{step p } v_0 \text{ s0 } t_0 = (v_1, \text{tg}, t_1) & \text{runT } v_1 \text{ s0 } t_1\ r\ b_1\ v_2\end{array}}{\text{runT } v_0 \text{ s0 } t_0\ r\ b_2\ v_2}\ \textsc{StepT}$$

$$\frac{\text{failed } t_0 \qquad \text{next\_alt } \bot\ t_0 = \bullet t_1 \qquad \text{runT } v_0 \text{ s0 } t_1\ r \text{ n } v_1}{\text{runT } v_0 \text{ s0 } t_0\ r \text{ n } v_1}\ \textsc{BackT}$$

$$\frac{\text{next\_alt } \bot\ t_0 = \square}{\text{runT } v_0 \text{ s0 } t_0\ \square\ \bot\ v_0}\ \textsc{FailT}$$

**Figure 7** Rule system for *runT*

## 3.4 Properties of *runT*

Working with *runT* allows to prove some interesting properties about the behavior of the cut operator wrt disjunction, i.e. the *Or* node. We have proven several properties that are avalable at *this* link. Below we show two of these lemmas.

$$\begin{array}{l}\textbf{Lemma}\ \texttt{runSST\_or}:\ \forall\ \texttt{p}\ v_0\ v_1\ \texttt{s0 s1}\ t_0\ t_0\texttt{'}\ \texttt{sm}\ t_1,\\ \quad \texttt{runT p}\ v_0\ \texttt{s0}\ t_0\ \bullet(\texttt{s1},\ \bullet t_0\texttt{'})\ \top\ v_1 \to\\ \qquad \texttt{runT p}\ v_0\ \texttt{s0 (Or}\ \bullet t_0\ \texttt{sm}\ t_1\texttt{)}\ \bullet(\texttt{s1},\ \bullet(\texttt{Or}\ \bullet t_0\texttt{' sm KO}))\ \bot\ v_1.\end{array} \tag{16}$$

$$\begin{array}{l}\textbf{Lemma}\ \texttt{runNT\_or}:\ \forall\ \texttt{p}\ v_0\ v_1\ \texttt{s0}\ t_0\ \texttt{sm}\ t_1,\\ \quad \texttt{runT p}\ v_0\ \texttt{s0}\ t_0\ \square\ \top\ v_1 \to\\ \qquad \texttt{runT p}\ v_0\ \texttt{s0 (Or}\ \bullet t_0\ \texttt{sm}\ t_1\texttt{)}\ \square\ \bot\ v_1.\end{array} \tag{17}$$

$$\begin{array}{l}\textbf{Lemma}\ \texttt{run\_orSST p}\ v_0\ v_2\ \texttt{s0 s1 sm}\ t_0\ t_0\texttt{'}\ t_1\ t_1\texttt{'}:\\ \quad \texttt{runT p}\ v_0\ \texttt{s0 (Or}\ \bullet t_0\ \texttt{sm}\ t_1\texttt{)}\ \bullet(\texttt{s1},\ \bullet(\texttt{Or}\ \bullet t_0\texttt{' sm}\ t_1\texttt{'}))\ \bot\ v_2 \to\\ \quad \exists\ b,\ \texttt{runT p}\ v_0\ \texttt{s0}\ t_0\ \bullet(\texttt{s1},\ \bullet t_0\texttt{'})\ b\ v_2 \wedge t_1\texttt{'} = \textbf{if}\ b\ \textbf{then}\ \texttt{KO}\ \textbf{else}\ t_1.\end{array} \tag{18}$$

In Definition 16, we evaluate the tree $t_0$. This evaluation produces a success with signature $\sigma_1$ and a new tree $t'_0$. During the evaluation, a superficial cut in $t_1$ is traversed. This implies that the evaluation of $\bullet t_0 \vee_{s_m} t_1$ returns $\sigma_1$ as substitution, and the alternatives are $\bullet t'_0 \vee_{s_m} \texttt{KO}$. The right-hand side of the *Or* node is replaced with *KO* by $!_r$ .

In Definition 17, we are in a similar scenario: $t_0$ evaluates a superficial cut but, this time, produces a failure. This implies that the evaluation of $\bullet t_0 \vee_{s_m} t_1$ also fails.

```
Definition stepE v₀ t σ a gl ≔
  let (v₁, rs) ≔ bc p v₀ t σ in
  (v₁, save_alts a gl rs).
```

$$\frac{}{\text{runE } v_0\ ((\sigma_0, \varnothing) :: \text{a})\ \bullet(\sigma_0, \text{a})}\ \textsc{StopE} \qquad \frac{\text{runE } v_0\ ((\sigma_0, \text{g}) :: \text{ca})\ \text{r}}{\text{runE } v_0\ ((\sigma_0, (\text{cut, ca}) :: \text{g}) :: \_)\ \text{r}}\ \textsc{CutE}$$

$$\frac{\text{stepE } v_0\ \text{t}\ \sigma_0\ \text{a g} = (v_1, \text{bs}) \qquad \text{runE } v_1\ (\text{bs} +\!\!+\ \text{a})\ \text{r}}{\text{runE } v_0\ ((\sigma_0, (\text{call t}, \_) :: \text{g}) :: \text{a})\ \text{r}}\ \textsc{CallE} \qquad \frac{}{\text{runE}\ \_\ \varnothing\ \square}\ \textsc{FailE}$$

**Figure 8** Rule system for *runE*

Finally, in Definition 18, we evaluate $\bullet t_0 \vee_{s_m} t_1$. The evaluation succeeds, producing the result $\bullet(s_1, \bullet t_0' \vee_{s_m} t_1')$. This means that $t_0$ still has $t_0'$ as unexplored alternatives. We deduce that the evaluation of $t_0$ produces the substitution $\sigma_1$ with alternatives $t_0'$. However, we have no information about whether a superficial cut has been crossed in $t_0$. If this is the case, we know that $t_1'$ is cut away by $!_r$ ; otherwise, $t_1'$ has not been explored and is equal to $t_1$.

From these lemmas, it is clear how the order in which alternatives are evaluated changes the behavior of the program.

se riesco success (A or B) = success (B or A) if no sup cut in A and B

## 4 Stack semantics

We now introduce the Elpi semantics. The interpreter is close to that of the Elpi language and provides an operational semantics that is similar to the one proposed by Pusch in [16], which is in turn closely related to the semantics given by Debray and Mishra in [6, Section 4.3]. Pusch mechanized this semantics in Isabelle/HOL, together with several optimizations that are also present in the Warren Abstract Machine [20, 1].

We represent a state of the Elpi language thanks to the two mutual inductives shown below.

```
Inductive alts  ≔ nilA | consA of (Σ * goals) & alts
with      goals ≔ nilG | consG of (A * alts)  & goals .
```

An Elpi state is an enhanced two-dimensional list. The outermost list represents a list of alternatives in disjunction, each accompanied by the substitution that should be used for their interpretation. The innermost list is a list of atoms, that is, the list of goals in conjunction. These goals are decorated with a pointer to an alternative list, which is used to keep track of where to jump when a cut is interpreted. We call these special alternatives the *cut-to* alternatives.

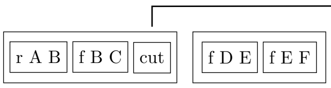
The Elpi interpreter is called *runE* and has the following type.

```
Inductive runE : 𝓕ᵥ → alts → option (Σ * alts) → Prop ≔        (19)
```

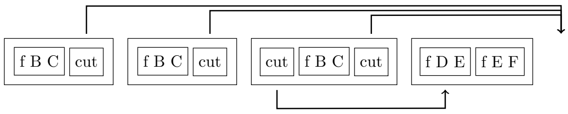
The inductive *runE* represents a routine taking as input a set of variables and a list of alternatives and returning an option. $\square$ stands for interpretation with no solution, whereas $\bullet(\sigma, a)$ represent a successful interpretation with $\sigma$ as solution and $a$ as the list of non-yet-explored choice points.

Dire che non abbiamo le due info ritornate da runT

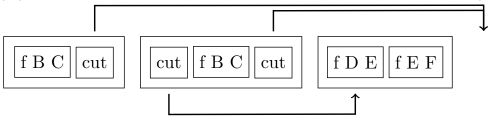
The rule system for *runE* is given in Figure 8 and is made by 4 cases. We distinguish two main cases: if the list of alternatives is empty, (FailE) we have a failure: we stop the computation and return $\square$. If the list of alternatives is the list $(\sigma_0, h) :: a$, the interpreter



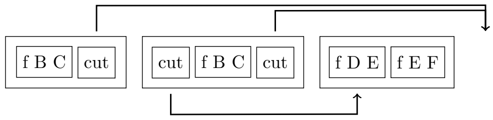

**(a)** Elpi state after call to `g 0 R`



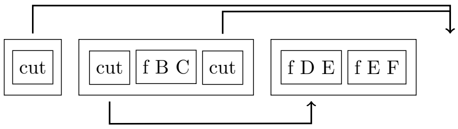

**(b)** Elpi state after evaluating Figure 9a




**(c)** Elpi state after evaluating Figure 9b




**(d)** Elpi state after evaluating Figure 9c




**(e)** Elpi state after evaluating Figure 9d

**Figure 9** Example of Elpi execution

evaluates the first choice point $h$ under the substitution $\sigma_0$. If $h$ is empty, (StopE), then we have a success, i.e. we return the input $\sigma_0$ and leave $a$ as the future choice points. Otherwise, $h$ is the list $(t, ca) :: g$ where $t$ is the first atom to evaluate, $ca$ is its cut-alternative list and $g$ is the list of future conjuncts. If $t$ is a cut (CutE), we need to keep evaluating $g$ under $\sigma_0$ but we change the alternatives to $ca$. Finally, in the case of a call (CallE), we backchain the call in the program. If $rs$ is the list of rule premises returned by `bc`, then each list of premises if mapped to a list of goals (`goals`) have $a$ as cut-alternative and the global list is translated to a list of alternative (`alts`).

Dire una parola su backtracking = tl if bc = nil add subst to ..

*Example of execution* We propose an exemple of execution of an Elpi state in Figure 9. The arrows in the images represent the cut-alternative pointer of the cut. We have not put arrow goiing our from atom-calls since the interpreter ignores the cut-alternatives in this case (cf. CallE).

*Remark* We want finally to point out how the lemma that we were able to prove in Section 3.4 are not easy to be expressed in the Elpi semantics. This is beacuse there is no sufficient structure in the Elpi state to understand what is the scope of the cut operator. For example, in an Elpi state made of these alternatives $a_0, a_1 \dots a_n$, we have no clear idea of what happens if the first alternative $a_0$ succeeds: there could be a cut in $a_0$ but, is this cut superficial? What is its impact wrt the alternatives $a_1 \dots a_n$? More then that, it is also possible that the state is hill-formed and the cut does not point to a suffix in the list of alternatives.

## 5 Equivalence of the two semantics

In the previous section we have introduces the two semantics, they are The equivalence between the two semantics is possible under two conditions: we need to work with "valid tree", i.e. all of those trees that can be generated from a call. Secondly, we need to translate trees into elpi states. In the next to subsection we propose the two functions *valid_state* and *t2l*.

```
Fixpoint valid_tree A :=
  match A with
  | TA _ | OK | KO ⇒ ⊤
  | Or □ _ B ⇒ valid_tree B
  | Or •A _ B ⇒ valid_tree A &&
        ((B == KO) || B.base_or B)
  | And A B0 B ⇒ valid_tree A &&
      if success A then valid_tree B
      else B == big_and B0
  end.
```

**(a)** Valid tree

```
Fixpoint t2l t₀ σ₀ (cp: alts) : alts :=
  match t₀ with
  | OK     ⇒ [(σ₀, ∅)]
  | KO     ⇒ ∅
  | TA a   ⇒ [(σ₀, [(a, ∅)])]
  ...
```

**(b)** Tree to list

**Figure 10** Valid tree and Tree to list

## 5.1 Valid trees (*valid_state*)

The inductive tree allows one to generate a large number of trees, some of which are not valid, in the sense that they cannot be produced starting from a given query. The class of valid trees is characterized by the function shown in Figure 10a.

While all base cases of tree are considered valid, we need to analyze carefully the cases for the *Or* and *And* constructors.

For the *Or* constructor, we distinguish two cases depending on whether the lhs exists. If it does not exist, then the ths must be a valid tree. Otherwise, the lhs must itself be a valid tree, and the rhs is either the *KO* tree, since it may have been removed by the evaluation of a superficial cut in the lhs, or it has not yet been explored. In the latter case, it is a `base_or` tree, namely the right-skewed tree formed by a disjunction of conjunctions that is genereated after a succeful backchain to a call.

For the *And* constructor, the lhs is required to be a valid tree. The shape of the rhs depends on whether the lhs is a success. If the lhs is not successful, then the rhs has never been explored: the procedures *step* and *next_alt* modify the rhs only when the lhs succeeds. In this case, the lhs must be the right-skewed tree containing conjunctions of premises atoms, the reset point $B_0$ ensure what shape the rhs should have. If the lhs is a success tree, then the rhs can be modified by *step* and *next_alt*, therefore it must be a valid tree.

## 5.2 From trees to lists (*t2l*)

The translation of a tree to a en ELPI state is done thanks to the *t2l* function. In Figure 10b we give junst the implementation of the base cases, the full implementation is available at *this* link.

The function *t2l* takes the tree $t_0$ we want to translate, a subsitution $\sigma_0$ and a list of choice points *cp*. These choice points represent alternatives that appears at the same level of the current node, such as, for example, the right-siblings of a *Or* node. The *OK* node represent one succeful alternative, thereofre it is translated into a singleton list with the empty list of goals. The *KO* node represent a failure, i.e. it is mapped to the empty list of alternatives. Finally, atoms are mapped to one alternative containing a single goal with the empty cut-alternative list. They cannot point to *cp* since they are not right-uncle subtrees.

We have omitted the code for the translation of the *Or* and *And* constructor since it is quite complex, but, we give the intuition behind this translation by means of examples.

The translation of $A \lor B_\sigma$ creates two list of alternatives, one for the $A$ and the other for

**Table 1** Tree to list from Figure 6 and Figure 9

| $t$ | Figure 6a | Figure 6b | Figures 6c and 6d | Figure 6e | Figure 6f |
|---|---|---|---|---|---|
| $t2l\ t\ \varepsilon\ \varnothing$ | Figure 9a | Figure 9b | Figure 9c | Figure 9d | Figure 9e |

$B$. The alternatives of the rhs are valid choice points for the lhs and should be passed to the translation of $A$. The two list of alternatives can be concatenated and, each atom, should add *cp* as new cut-to alternatives: $A$ and $B$ are one level lower wrt *cp* and therefore the cut they have should point to *cp*.

The translation of $A \wedge_{B_0} B$ implies the translation of $A$. If the translation of $A$ returns an empty list we return the empty list of alternatives without even touching $B$ nor $B_0$. Otherwise the translation returns a list $(\sigma_0, g) :: a$. By the semantics of *runT* the $B$ node represent the continuation of the first alternatives in $A$, i.e. $g$, whereas $B_0$ is the continuation of goals for each alternative in $a$. While performing these translation, we need to carefully update the cut-atlernatives.

dirlo meglio all'inizio

*Example of* t2l In Table 1 we point out the result of calling *t2l* on the trees in Figure 6. The first raw contains the image reference of then trees and the second raw contains the reference of the images from the ELPI state in Figure 9. For the $3^{rd}$ column in the table, we see that *t2l* is not injective. There are different trees that maps to the same ELPI state. In fact, there are transitions in *runT* that are not visible in *runE*.

## 5.3 Equivalence theorems

Thanks to the two functions *valid_state* and *t2l*, we are able to finally state the equivalence theorems between the semantics. The proof we are giving are on a slightly different definition of *runT*, since in the equivalence, we ignore the verification for the boolean and the variable set returned by *runT*, these piece of data are paricularly meant for the proof in Section 3.4.

The definition we use is the following:

```
Definition runT' p v σ t r := ∃ b fv, runT p v σ t r b fv.
```
(20)

Before going to the equivalence proofs, we want to claim a key lemma allowing to ignore the silent transition in the *runT* inductive.

```
Lemma next_altF_t2l: ∀ t t' b,
  valid_tree t → failed t → next_alt b t = •t' →
    ∀ l σ, t2l t σ l = t2l t' σ l.
```
(21)

It is proved easily by induction on the tree $t_0$.

dire di $v_0$

```
Lemma tree_to_elpi: ∀ p t σ r,
  let v := vars_tree t `|` vars_sigma σ in
  valid_tree t →
    runT' p v σ t r →
      let a := t2l t σ ∅  in
      let r' :=  if r is •(σ', t) then  •(σ', t2l (odflt KO t) σ ∅ )
                 else □ in
      runE p v a r'.
```
(22)

*Proof sketch* We proceed by induction on the execution of *runT'*. There are 4 cases to be treated. The first case comes from STOPT, it deals with successful trees, a successful tree must to start with an empty list and therefore we conclude with STOPE. The case for STEPT, need to treat two cases: *step* has evaluated either a cut or a call. Depending on this we should use CUTE or CALLE and then the induction hypothesis. The case for BACKT is a silent case: it represents the non-injective behavior of *t2l*, but, thanks to Definition 21 and

$$
\begin{tikzcd}
t_0 \arrow[r, "runT\text{'}\ t_0"] \arrow[d, "\text{t2l}"'] & t_1 \arrow[d, "\text{t2l}"] \\
a_0 \arrow[r, "runE\ a_0"'] \arrow[ur] & a_1
\end{tikzcd}
$$

**Figure 11** Induction scheme for Definition 23

the induction hypotheses the conclusion ist proved. The last case comes from the derivation FAILT. It is easily proved using FAILE and the fact that if *next_alt* returns ☐ when trying to earse failure in $t_0$, then *t2l* of $t_0$ returns the empty list.

```
Lemma elpi_to_tree p v0 a0 r0 :
  runE p v0 a0 r0 →
  ∀ s0 t0, valid_tree t0 → t2l t0 s0 ∅ = a0 →
  if r0 is •(s1, a1) then
    ∃ t1, runT' p v0 s0 t0 •(s1, t1) ∧ t2l (odflt KO t1) s0 ∅ = a1
  else runT' p v0 s0 t0 ☐.
```
(23)

The proof of Definition 23 is based on the idea explained in [2, Section 3.5.1]. An ideal statement for this lemma would be: given a function *l2t* transforming an ELPI state $a_0$ to a tree $t_0$, we then would be able to compare the result of *runE* run with $a_0$ and *runT'* run with $t_0$. The function it is difficult to implement since we have hard time finding the tree structure of $a_0$, and more than that, we should have a notation of ELPI valid state, but again, this is a hard task. We thereofre need to reuse the *t2l* function to relate a tree $t_0$ with $a_0$. Figure 11 show the idea of the proof: we have a state $t_0$ whose translation is $a_0$, we proceed by induction on the execution of *runE*. This gives us not only the output $a_1$ but that hypothesis that it exists a tree $t_1$ such that its translation to the ELPI state is $a_1$. This proof strategy is indirectly providing a kind of *l2t* function by combining the execution of the *runE* and *t2l*.

## 6 Case study: determinacy alanysis

we mechanize the first order part of xxx.

snippet det, main thm, invariant det tree (valid tree prev section?)

proof induciton on exec, step/next alt preserving invariant proved by induction on the tree.

with list semantics cut and next alt requires to express a link btween the ca or next alts and the current goal, which is non trivial without an intermediate data strature like the tree

## 7 Related work

The input mode of Elpi, inspired by the XXXX of BProlog, has only an impact in the determinacy analysis that does not required to be preceeded by a mode analysis. Well moded program propagate the groundness from input arguments to output arguments. As a result if the initial query is ground then unification degenerates to matching (for input arguments). Considering a semantics where input arguments are match has the following impact on det analysis: axiom XXX does not require q to be ground.

The only mechanization of Prolog's semantics we could find is the one by Pusch in ISABELLE/HOL [16] that is based on the model of Debray and Mishra [6, Sec. 4.3]. Their work start from that semantics and refines it by introducing some optimizations usually found in the Warren abstract machine [20, 1]. They do not use the semantics to prove properties on the execution of programs as we do in our use case, hence they do not face the difficulty described in seciton XXX that pushed us to develop an more explicit semantics (with respect to cut). In some way our work is complementary, showing he stack based semantics equivalent to more optmized one, we could consider implement in Elpi.

Another relevant work are the ones by King cite, but we could not find their code. Their semantics is denotational and cannot easily cover all the features our paper XX does, but would have been sufficient for this first step.

The proof technique we use to relate the two semantics is inspired by XXX and we thank Y.Bertot for insightful discussions on the topic. In XXX Bertot relates bla bla, witout never constructing a decompiler, exactly as we never build a list to tree function.

## 8 Conclusion

We do this. This is a first step for XXXX. We need to add substitutions. The missing part of the proof becomes related to abstract interpretation, since the det types for a lattice and one has to prove that the concrete runtime values variables take agree with the abstraction computed statically. This proof is ongoing and somehow orthogonal to the current one since the cut operator and the HO feature do not interfere with eachother in complex ways.

## References


1. Hassan Aït-Kaci. *Warren's Abstract Machine: A Tutorial Reconstruction*. The MIT Press, 08 1991. `doi:10.7551/mitpress/7160.001.0001`.
2. Yves Bertot. A certified compiler for an imperative language. Technical Report RR-3488, INRIA, September 1998. URL: `https://inria.hal.science/inria-00073199v1`.
3. Valentin Blot, Denis Cousineau, Enzo Crance, Louise Dubois de Prisque, Chantal Keller, Assia Mahboubi, and Pierre Vial. Compositional pre-processing for automated reasoning in dependent type theory. In Robbert Krebbers, Dmitriy Traytel, Brigitte Pientka, and Steve Zdancewic, editors, *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2023, Boston, MA, USA, January 16-17, 2023*, pages 63–77. ACM, 2023. `doi:10.1145/3573105.3575676`.
4. Cyril Cohen, Enzo Crance, and Assia Mahboubi. Trocq: Proof transfer for free, with or without univalence. In Stephanie Weirich, editor, *Programming Languages and Systems*, pages 239–268, Cham, 2024. Springer Nature Switzerland.
5. Cyril Cohen, Kazuhiko Sakaguchi, and Enrico Tassi. Hierarchy Builder: Algebraic hierarchies Made Easy in Coq with Elpi. In *Proceedings of FSCD*, volume 167 of *LIPIcs*, pages 34:1–34:21, 2020. URL: `https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2020.34`, `doi:10.4230/LIPIcs.FSCD.2020.34`.
6. Saumya K. Debray and Prateek Mishra. Denotational and operational semantics for prolog. *J. Log. Program.*, 5(1):61–91, March 1988. `doi:10.1016/0743-1066(88)90007-6`.
7. Cvetan Dunchev, Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. ELPI: fast, embeddable, λProlog interpreter. In *Proceedings of LPAR*, volume 9450 of *LNCS*, pages 460–468. Springer, 2015. URL: `https://inria.hal.science/hal-01176856v1`, `doi:10.1007/978-3-662-48899-7\_32`.
8. Davide Fissore and Enrico Tassi. A new Type-Class solver for Coq in Elpi. In *The Coq Workshop*, July 2023. URL: `https://inria.hal.science/hal-04467855`.

**9** Davide Fissore and Enrico Tassi. Higher-order unification for free!: Reusing the meta-language unification for the object language. In *Proceedings of PPDP*, pages 1–13. ACM, 2024. `doi:10.1145/3678232.3678233`.

**10** Davide Fissore and Enrico Tassi. Determinacy checking for elpi: an higher-order logic programming language with cut. In *Practical Aspects of Declarative Languages: 28th International Symposium, PADL 2026, Rennes, France, January 12–13, 2026, Proceedings*, pages 77–95, Berlin, Heidelberg, 2026. Springer-Verlag. `doi:10.1007/978-3-032-15981-6_5`.

**11** Benjamin Grégoire, Jean-Christophe Léchenet, and Enrico Tassi. Practical and sound equality tests, automatically. In *Proceedings of CPP*, page 167–181. Association for Computing Machinery, 2023. `doi:10.1145/3573105.3575683`.

**12** Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. Implementing type theory in higher order constraint logic programming. In *Mathematical Structures in Computer Science*, volume 29, pages 1125–1150. Cambridge University Press, 2019. `doi:10.1017/S0960129518000427`.

**13** Robbert Krebbers, Luko van der Maas, and Enrico Tassi. Inductive Predicates via Least Fixpoints in Higher-Order Separation Logic. In Yannick Forster and Chantal Keller, editors, *16th International Conference on Interactive Theorem Proving (ITP 2025)*, volume 352 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 27:1–27:21, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: `https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2025.27`, `doi:10.4230/LIPIcs.ITP.2025.27`.

**14** Dale Miller. A logic programming language with lambda-abstraction, function variables, and simple unification. In *Extensions of Logic Programming*, pages 253–281. Springer, 1991.

**15** Dale Miller and Gopalan Nadathur. *Programming with Higher-Order Logic*. Cambridge University Press, 2012.

**16** Cornelia Pusch. Verification of compiler correctness for the wam. In Gerhard Goos, Juris Hartmanis, Jan van Leeuwen, Joakim von Wright, Jim Grundy, and John Harrison, editors, *Theorem Proving in Higher Order Logics*, pages 347–361, Berlin, Heidelberg, 1996. Springer Berlin Heidelberg.

**17** Enrico Tassi. Elpi: an extension language for Coq (Metaprogramming Coq in the Elpi $\lambda$Prolog dialect). In *The Fourth International Workshop on Coq for Programming Languages*, January 2018. URL: `https://inria.hal.science/hal-01637063`.

**18** Enrico Tassi. Deriving proved equality tests in Coq-Elpi. In *Proceedings of ITP*, volume 141 of *LIPIcs*, pages 29:1–29:18, September 2019. URL: `https://inria.hal.science/hal-01897468`, `doi:10.4230/LIPIcs.CVIT.2016.23`.

**19** Luko van der Maas. Extending the Iris Proof Mode with inductive predicates using Elpi. Master's thesis, Radboud University Nijmegen, 2024. `doi:10.5281/zenodo.12568604`.

**20** David H.D. Warren. An Abstract Prolog Instruction Set. Technical Report Technical Note 309, SRI International, Artificial Intelligence Center, Computer Science and Technology Division, Menlo Park, CA, USA, October 1983. URL: `https://www.sri.com/wp-content/uploads/2021/12/641.pdf`.